

BCSE204L Design and Analysis of Algorithms

Module-1

Topic-1: Basics of Algorithms

Dr Sunil Kumar P V

SCOPE

Algorithm

- Informally, an algorithm is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.
- An algorithm is thus a sequence of computational steps that transform the input into the output
- A tool for solving a well-specified computational problem.
- Any particular input is called an instance
- An algorithm is said to be correct if, for every input instance, it halts with the correct output.
- An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer
- Examples??

Why to study algorithms?

- Computers are not infinitely fast and memory is not free
- If computers were infinitely fast, any correct method for solving a problem would do.
- Computers may be fast, but they are not infinitely fast.
- And memory may be inexpensive, but it is not free
- Computing time and memory are therefore bounded resources
- These resources are to be used wisely
- Algorithms that are efficient in terms of time or space will help with that.

Stages of Algorithm Development

1. Describing the problem
2. Identifying a suitable technique
3. Design of an algorithm
4. Proof of Correctness of the algorithm

Describe the problem & Find a method

- Example:- Finding the average of list of n numbers
- Input:- The list of n numbers
- Output:- The average
- Method:-
 - Sum all elements and store in sum
 - Divide the sum by n which is the average. Store it in avg
 - Output the Average avg

Design an algorithm

- Can be done in English, pseudocode, flowchart so on..
 1. Receive n as input
 2. Receive n numbers as input
 3. Find the sum of n numbers and store in a variable sum
 4. Compute $avg = \frac{sum}{n}$
 5. Output avg

Correctness of Algorithm

- Correct:- if, for every input instance, it halts with the correct output
- An incorrect algorithm
 - may not halt at all for some input instances, or
 - may halt with a wrong output
- Correct if the algorithm yields the desired output for the specific input in a finite amount of time.
- Proof of correctness depends on the algorithm

Correctness of Algorithm [cont..]

- For some algorithms, the proof of correctness is easy and for some algorithms, it is complex
- It depends on the type of instructions in the algorithm
- In order to show that an algorithm is incorrect, one instance of input is sufficient for which the algorithm fails to give the desired output

Proof of correctness

- Explanation by example
- Consider the insertion sort algorithm

INSERTION-SORT(A)

```
1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1 \dots j - 1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i + 1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i + 1] = key$ 
```

Loop invariant method

- A loop invariant is a property of a program loop that is true before (and after) each iteration
- In our insertion sort, the loop invariant is:
 - At the start of each iteration of the for loop of lines 1–8, the subarray $A[1 \cdots j - 1]$ consists of the elements originally in $A[1 \cdots j - 1]$, but in sorted order.

Loop Invariant method [cont..]

- In loop invariant method of proving correctness, one must show three things about a loop invariant:
 - Initialization: It is true prior to the first iteration of the loop.
 - Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration.
 - Termination: When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct

Initialization

- To show that the loop invariant holds before the first loop iteration
- When $j = 2$, the subarray $A[1 \cdots j - 1]$ consists of just the single element $A[1]$
- Which is the original element in $A[1]$
- This subarray is trivially sorted

Maintenance

- The body of the for loop works by moving $A[j - 1]$, $A[j - 2]$, $A[j - 3]$, and so on by one position to the right until it finds the proper position for $A[j]$ (lines 4–7), at which point it inserts the value of $A[j]$ (line 8).
- The subarray $A[1..j]$ then consists of the elements originally in $A[1..j]$, but in sorted order.
- Incrementing j for the next iteration of the for loop then preserves the loop invariant

Termination

- The condition causing the for loop to terminate is that $j > A.length = n$
- Substituting $n + 1$ for j in the wording of loop invariant, we have that the subarray $A[1..n]$ consists of the elements originally in $A[1..n]$, but in sorted order
- As $A[1..n]$ is the original array it implies that the entire array is sorted
- Hence the algorithm is correct

Exercises

1. Write the iterative algorithm to determine the sum of first n natural numbers. Prove its correctness using loop invariant method
2. Write the iterative algorithm to determine the product of first n natural numbers. Prove its correctness using loop invariant method.

Deriving Time Complexity

- Asymptotic notations
- Frequency counting method
- Recurrence tree method
- Substitution method
- Master's method

Thank you...!!